

Guardian AI

Project Progress Report

AI-Powered RF + Vision Intelligence for Exam-Hall Security

22 July 2026

Internal team distribution

Phases 1-4 | Repository state at commit 6648f7d

1. Executive Summary

Guardian AI is an RF intelligence platform whose first application is exam-hall security: passively detecting hidden wireless devices and correlating them with people, without intercepting any communication. The project follows a simulation-first pipeline - Digital Twin, then synthetic data, then AI training, then real deployment - so that each layer can be validated before hardware exists.

Where the project stands. Phase 2 (Digital Twin) is complete and now has its generator scripts under version control. Phase 3 (RF Intelligence) has a trained localization model that clears both of its frozen targets on simulated data, but is blocked awaiting USRP B210 hardware. Phase 4 (Vision AI) went from not started to a working, measured pipeline during this reporting period. Phase 5 (Fusion and Platform) has not begun.

Headline results. RF localization reaches 0.0 m median error and a 94.9% correct-bench rate against the simulated dataset. Vision geometry reaches 100% correct-seat assignment with 0.160 m median position error across 201 synthetic people. Both figures come from simulation, not the field, and this report is explicit throughout about what each number does and does not establish.

What needs attention. Two findings carry beyond software. First, 15 of the 99 seats are outside the documented camera's field of view - the front rows sit almost directly beneath it - which caps achievable vision coverage at 84.8% regardless of model quality. Second, both RF and Vision are now blocked on physical inputs, which makes Phase 5 the only track that can advance without new hardware.

2. Project Scope and Conventions

2.1 Frozen Phase 1 scope

Scope creep is the project's named top risk, so Phase 1 is deliberately frozen to: the 2.4 GHz ISM band only, a single room, RSSI-based localization, and BLE versus Wi-Fi classification. Anything outside that boundary - 5/6 GHz, Zigbee, LoRa, cellular, TDoA/AoA, full behaviour AI, or expansion beyond exam halls - is recorded in ROADMAP.md rather than absorbed into current work.

A privacy constraint applies across every track: sensing is passive only, captured signals are never demodulated, and a data-retention policy must exist before any pilot deployment. That policy is still unwritten and is flagged in Section 7.

2.2 Track ownership

Track	Responsibility	Primary stack
Digital Twin	3D hall model, seat map, node placement, synthetic data	Blender, Python
RF Intelligence	Signal capture, protocol classification, RSSI localization	GNU Radio, Python, USRP B210
Vision AI	Person detection, pixel-to-hall mapping, seat assignment	YOLOv8, OpenCV, PyTorch

The Digital Twin track leads by design: it produces the coordinate system, seat map, and node placement that every other track depends on.

2.3 Shared conventions

All tracks share one coordinate system - origin at the front-left corner, X across rows, Y along rows, metres - and one seat identifier format, Row-Section-Seat (for example R06-C03). Every module emits the same JSON event schema from day one, so that Phase 5 fusion has a single contract to consume:

```
event_id | timestamp_utc | source_module (rf/vision/localization/fusion)
event_type (signal_detected/person_detected/position_estimate/alert)
position {x, y, seat, error_m} | signal {protocol, freq_hz, rssi_dbm, ...}
confidence | evidence_ref
```

Fields that do not apply to a given module are present and explicitly null rather than omitted. The vision module, for example, sets position.error_m to null because no ground truth exists at inference time to measure error against.

3. Phase Status

Phase	Track	Status	Notes
1 - Scope definition	All	Complete	Frozen; deviations go to ROADMAP.md
2 - Digital Twin	Digital Twin	Complete	Hall, seat map, RF simulation, 39,600-sample dataset; generators now committed
3 - RF Intelligence	RF	Blocked	Model trained and passing on simulated data; awaiting USRP B210 capture
4 - Vision AI	Vision	In progress	Pipeline built and measured on synthetic data; awaiting real footage
5 - Fusion and Platform	All	Not started	Can proceed without hardware - see Section 8

4. Work Completed This Period

Four commits, in dependency order. Each was reviewed before commit using the project's own review agents, and every fix listed below came out of that process.

4.1 Digital Twin generator scripts brought under version control (4496524)

The three scripts that produced every Phase 2 artefact - the Blender hall builder, the RF node-placement optimizer, and the RF propagation simulator - existed only as loose local files. Their outputs were committed; the code that generated them was not. They now live in DigitalTwin/scripts/ with three correctness fixes:

- Hardcoded Desktop input and output paths replaced with repository-relative ones, removing a manual copy step that invited silent drift between a rerun and what was committed.
- Random seeds fixed, so reruns are deterministic and diffable rather than merely statistically similar.
- The RF simulator no longer hardcodes the winning four-corner node layout. It reads the optimizer's own best_node_placement.json output, with a fail-fast check if the node count ever stops matching - previously the two files could silently disagree if the optimizer were rerun.

Both plain-Python scripts were rerun end to end to confirm they still select the four-corner layout and still reproduce the 39,600-sample dataset shape. The regenerated files were then reverted, so the committed training data - which the existing model was trained on - was not silently replaced.

4.2 Vision AI pipeline scaffolded (63f3c68)

Phase 4 began from an empty directory. The pipeline mirrors the existing AI/localization module's structure and testing conventions:

Module	Responsibility
detection.py	Person detection via YOLOv8, behind an injectable backend
homography.py	Pixel to hall-coordinate mapping (cv2.findHomography)
seat_mapping.py	Nearest-seat lookup against Shared/seat_map.json
events.py	person_detected events in the shared schema
pipeline.py	Wires detect, project, seat lookup and event emission together

The detector's backend is dependency-injected specifically so the test suite never imports ultralytics or torch; the real YOLO model is loaded lazily and only in production paths. This keeps the suite fast and runnable in environments where those heavy dependencies are not installed.

4.3 Synthetic vision dataset generated (588e3cc)

Vision had no data of any kind. Unlike the RF track, which inherited a full simulated dataset from Phase 2, the Digital Twin had only ever emitted JSON specifications - no imagery. Two generators now close that gap, both emitting an identical ground_truth.json schema:

- **project_vision_dataset.py** - pure pinhole-camera projection in NumPy. Needs no Blender, so it runs and can be verified anywhere Python does. This produced the committed dataset.
- **render_vision_dataset.py** - the Blender equivalent, which additionally renders PNG imagery for when a real detector needs real pixels. It also adds a genuine camera to the scene: the hall builder had only ever created a decorative cube, so the model contained nothing to render from.

Both choose the camera's downward pitch by measurement rather than assumption - sweeping candidate angles and keeping whichever sees the most seats, the same approach the RF node optimizer uses to choose a layout. The selected pitch is 20 degrees below horizontal.

4.4 Vision geometry evaluated (6648f7d)

An evaluation module mirroring AI/localization's data / evaluate / report / train split now produces Vision's first real metrics, written to Vision/perception/results/ as metrics.json, a text report, a per-person CSV, and three figures.

Detection is deliberately not exercised. The synthetic people are bare geometric primitives - a cylinder and a sphere - which a YOLO model trained on real photographs would almost certainly fail to detect. Running it would measure that domain gap rather than the pipeline. Instead each person's exact ground-truth pixel box is fed through the real production pipeline, isolating one question: does the homography and seat-mapping chain recover the seat the person was actually placed in? A documented seam exists to swap in the real detector once genuine footage exists.

5. Measured Results

5.1 RF localization (simulated)

Two k-nearest-neighbour models were trained on the Phase 2 simulated RSSI dataset (39,600 samples: 99 seats x 4 nodes x 100 readings). Readings are averaged over a 40-sample observation window, because a single instantaneous RSSI value carries roughly 3 dB of noise while neighbouring seats differ by only 1-2 dB - too little to separate reliably from one reading.

Model	Median err.	Mean err.	Correct bench	Correct seat
FingerprintKNN (selected)	0.000 m	0.125 m	94.9%	89.9%
RegressorKNN	0.527 m	0.599 m	84.8%	73.7%

Both frozen targets are met by the selected model: median error below 2 m, and correct-bench rate above 80%. Evaluated on 99 held-out test observations.

5.2 Vision geometry (synthetic)

Metric	Result	Interpretation
Frame coverage	84 / 99 seats (84.8%)	Physical camera limitation, not a software one
Correct seat	100.0% (n = 201)	Geometry chain resolves the right seat
Median position error	0.160 m	Equals the placeholder's body radius
Mean position error	0.163 m	Consistent with the median - no long tail
Max position error	0.391 m	Well inside the 0.53 m seat spacing
Single occupancy	100.0% (n = 84)	One person per frame
Multi occupancy	100.0% (n = 117)	3-12 people per frame; no degradation

How to read these numbers. They are geometry-only. They measure the homography and seat-mapping chain given perfect pixel input, and include no detection error whatsoever. They are therefore *not* the project's 90% vision seat accuracy target, which requires a detector working on real footage. This caveat is written into every generated output file so it cannot be quoted out of context later.

A second limitation is worth stating plainly: this is a closed loop. The same camera model both generates the data and is inverted to evaluate it, so the result demonstrates internal consistency rather than real-world robustness. It cannot account for lens distortion, mounting error, or calibration drift. It is still a meaningful check - it caught a genuine defect described in Section 7 - but it is a floor on correctness, not evidence of field readiness.

5.3 Success metrics against frozen targets

Metric	Target	Current status
Detection rate	> 90%	Not measured - needs RF hardware
Classification rate	> 90%	Not measured - needs RF hardware
Localization median error	< 2 m	0.0 m (simulated) - target met
Correct-bench rate	> 80%	94.9% (simulated) - target met
Vision seat accuracy	> 90%	Geometry-only proxy 100%; end-to-end not measured
Alert latency	< 5 s	Not measured - needs Phase 5
False alerts	< 1 / hour	Not measured - needs Phase 5

Two of seven metrics are met on simulated data. The remaining five depend on hardware or on Phase 5 existing.

6. Verification and Quality

Suite	Tests	Covers
AI/tests	12 passing	Data loading, models, evaluation, event schema
Vision/tests	30 passing	Homography, seat mapping, detection filtering, pipeline wiring, dataset loading, evaluation

Every test uses small hand-built fixtures or stub objects rather than mocking frameworks, matching the conventions already established in the repository. Failure cases are covered alongside happy paths: empty seat maps, insufficient homography points, degenerate projections, mismatched input lengths, and absent datasets all have explicit tests.

The two suites must be run separately. Both directories contain a file named `test_events.py`, and with no packaging configuration in the repository, a combined invocation fails on the duplicate module basename. This is pre-existing and worth resolving before a CI pipeline is introduced.

7. Findings, Defects and Risks

7.1 Fifteen seats are outside the camera's field of view

The seat map asserts that the camera *covers all 11 rows, all 99 seats*. Measurement contradicts this. At the best available pitch, 84 of 99 seats fall inside the frame; the front rows sit almost directly beneath the ceiling-mounted camera and cannot be seen from it.

Action required, and it is not a software fix. Achievable vision coverage is capped at 84.8% until a second camera is added or the existing one is moved or re-aimed. This is a Digital Twin decision. The seat map's coverage claim should also be corrected so downstream work stops inheriting an untrue assumption. See `seat_coverage_map.png` in the results package.

7.2 Homography must be calibrated on the bench plane

The calibration was initially fitted on the floor plane ($z = 0$). A seated person's lowest visible point rests on the bench surface at $z = 0.45$ m, not the floor. Projecting a bench-height point through a floor-calibrated homography extends it along its view ray until it reaches the floor, displacing it away from the camera by

roughly 20% of its distance - about 2.4 m at the back row, or two full rows of error.

Both generators now calibrate on the bench plane. This single change is the difference between a 0.160 m median error and an unusable result, and it is the clearest argument for having built the verifiable pure-Python generator: the defect was predicted analytically, then confirmed and corrected by execution.

7.3 Defects caught in review

- **Renderer visibility (critical).** The Blender script toggled render visibility on a parent object, but that property does not cascade to children in Blender. Every placeholder body would have appeared in every frame, contaminating the entire image set with unlabelled figures. Because the script cannot be executed in the current environment, this would otherwise have surfaced only on visual inspection of the output.
- **Silent result misalignment.** The evaluator paired people to pipeline events positionally. Any future change that dropped an event would have produced plausible-looking but wrong accuracy figures - the most damaging possible failure for a metrics deliverable. It now fails loudly on a count mismatch.
- **Configuration drift.** The RF simulator duplicated the node layout the optimizer had selected, with nothing linking them. It now reads the optimizer's output directly.

7.4 Standing risks

Risk	Impact	Position
No RF hardware	High	Phase 3 cannot be validated; all figures simulated
No real footage	High	Vision detection accuracy remains unmeasured
Camera blind spot	Medium	Caps coverage at 84.8%; needs a placement decision
Retention policy absent	Medium	Required before any pilot; not yet drafted
Scope creep	Medium	Named top risk; controlled via frozen scope and ROADMAP.md

8. Recommended Next Steps

Both the RF and Vision tracks are now blocked on physical inputs. Phase 5 is the only substantial work that can proceed without new hardware, which makes it the natural priority.

#	Work	Track	Blocked?
1	Phase 5: fusion engine, event bus, FastAPI and dashboard scaffolding - buildable now against the shared event schema using synthetic events	All	No
2	Draft the privacy and data-retention policy required before any pilot	All	No
3	Resolve the camera blind spot: add, move or re-aim a camera, and correct the seat map's coverage claim	Digital Twin	No
4	Capture real RSSI with the USRP B210 and revalidate the localization model against it	RF	Hardware
5	Obtain real or higher-fidelity footage; calibrate from real correspondences and measure true end-to-end vision accuracy	Vision	Footage
6	Add multi-frame tracking (ByteTrack) once a live frame stream exists	Vision	Footage

One cross-cutting note: the timing-based correlation strategy - matching an RF signal to a person by *when* it appears rather than by position alone - remains the core disambiguation idea for Phase 5, and nothing built so far constrains it.

Appendix A. Repository Layout

```
AI/ localization model, training data, tests, results
DigitalTwin/ generator scripts, RF simulation output, vision simulation output
Vision/ perception modules, synthetic dataset, tests, results
Shared/ seat_map.json - the canonical coordinate and seat reference
RF/ (awaiting Phase 3 hardware work)
Fusion/ Backend/ Dashboard/ (awaiting Phase 5)
Docs/ documentation
ROADMAP.md deferred ideas, kept out of frozen Phase 1 scope
status.md living status record
```

Appendix B. Reproducing the Results

```
# RF node placement and propagation simulation
python DigitalTwin/scripts/node_placement_optimizer.py
python DigitalTwin/scripts/rf_propagation_simulator.py

# Localization model training and evaluation
python AI/localization/train.py

# Vision synthetic dataset and geometry evaluation
python DigitalTwin/scripts/project_vision_dataset.py
python Vision/perception/train.py

# Tests (run separately - see Section 6)
python -m pytest AI/tests
```



```
python -m pytest Vision/tests
```

The Blender renderer is run separately and requires a Blender installation:

```
blender --background --python DigitalTwin/scripts/hall_builder.py \  
--python DigitalTwin/scripts/render_vision_dataset.py
```

Appendix C. Commit History

Commit	Description	Period
7a096a5	Initial repository scaffolding and project conventions	Earlier
f9160a9	Phase 2 Digital Twin deliverables: seat map, RF simulation, dataset	Earlier
6409aac	AI localization model: training pipeline, tests, results	Earlier
4496524	Digital Twin generator scripts integrated, with fixes	This period
63f3c68	Vision AI scaffold: detection, homography, seat mapping, events	This period
588e3cc	Synthetic vision dataset generated for Phase 4 evaluation	This period
6648f7d	Vision pipeline geometry evaluated against the synthetic dataset	This period

All figures in this report were read directly from the committed results packages at commit 6648f7d and are reproducible with the commands in Appendix B.